Montador RISC-V (Versão Simplificada)
CCF 252 Trabalho 1

Pablo da Silva Santana (5900)[1], **Joao Pedro De Oliveira Rodrigues (4425)**[2]

[1]Graduação de Informática – Universidade Federal de Viçosa, Campus Florestal (UFV)
Endereço: Rodovia LMG 818, km 6 - Florestal, Minas Gerais, CEP 35.690-000

pablo.s.santana@ufv.br, joao.rodrigues9@ufv.br

Abstract. *This paper presents the development of a simplified RISC-V assembler as part of the first practical assignment of the Computer Organization course (CCF-252) in the Computer Science undergraduate program at Universidade Federal de Viçosa, Campus Florestal. The project consists of implementing a tool capable of translating a subset of RISC-V assembly instructions into their corresponding binary representations. The assembler was developed using Python and is designed to run in a Linux environment, following the specifications provided in the assignment.*

Resumo. *Este artigo apresenta o desenvolvimento de um montador simplificado para a arquitetura RISC-V, como parte do primeiro trabalho prático da disciplina CCF-252 (Organização de Computadores), do curso de Ciência da Computação da Universidade Federal de Viçosa, Campus Florestal. O projeto consiste na implementação de uma ferramenta capaz de traduzir um subconjunto de instruções em assembly RISC-V para suas representações em código binário. O montador foi desenvolvido em Python e projetado para execução em ambiente Linux, seguindo as especificações propostas no trabalho.*

1. Informações Gerais

Este trabalho consiste na implementação de um montador simplificado para a arquitetura RISC-V, desenvolvido como parte da disciplina de Organização de Computadores. O objetivo principal do projeto é traduzir instruções escritas em linguagem assembly para sua representação binária correspondente, seguindo o padrão da arquitetura RISC-V.

O programa foi desenvolvido utilizando a linguagem Python, sendo projetado para execução em ambiente Linux. Não há necessidade de compilação prévia, bastando a utilização de um interpretador Python compatível.

O código-fonte do projeto está disponível em repositório público no GitHub, podendo ser acessado em: `https://github.com/jotinha156/Trabalho1-ccf-252`.

O montador implementa um subconjunto de instruções da arquitetura RISC-V, incluindo *add*, *or*, *sll*, *andi*, *lh*, *sh* e *bne*. Além disso, o sistema oferece suporte à utilização de rótulos (labels), permitindo a resolução de desvios em instruções do tipo branch.

A execução do programa pode ser realizada por meio da linha de comando do terminal, utilizando os seguintes formatos:

```
py montador.py entrada.asm
py montador.py entrada.asm -o saida.txt
```

No primeiro caso, as instruções convertidas são exibidas diretamente no terminal. No segundo, o resultado é armazenado em um arquivo de saída especificado pelo usuário.

O arquivo de entrada deve estar no formato `.asm`, contendo instruções partes do subconjunto do projeto e válidas pela arquitetura RISC-V. A saída gerada consiste em uma sequência de instruções em formato binário, com uma instrução por linha.

2. Instalação e Execução

O montador foi desenvolvido em Python, sendo necessário possuir o interpretador instalado em um ambiente Linux para sua execução.

Para instalar o Python em sistemas baseados em Ubuntu, utilize o seguinte comando:

```
sudo apt update
sudo apt upgrade
sudo apt install python3
```

Após a instalação, o programa pode ser executado diretamente via terminal. Para converter um arquivo de entrada `.asm`, utilize:

```
py montador.py entrada.asm
```

Nesse caso, as instruções convertidas serão exibidas no terminal.

Para salvar o resultado em um arquivo de saída, utilize:

```
py montador.py entrada.asm -o saida.txt
```

Neste formato, o código binário gerado será armazenado no arquivo especificado.

As figuras 1, 2 e 3, respectivamente, apresentam o arquivo de entrada, a execução do programa no terminal, e o arquivo de saida gerado.

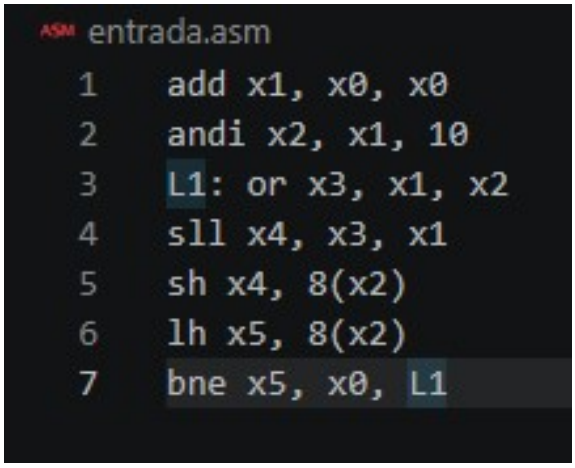

Figura 1. Arquivo de Entrada

```
TERMINAL    PORTS

\Trabalho1-ccf-252> py montador.py entrada.asm -o saida.txt
```

Figura 2. Execução no Terminal

```
saida.txt
1   00000000000000000000000010110011
2   00000000101000001111000100010011
3   00000000001000001110000110110011
4   00000000000100011001001000110011
5   00000000010000010001010000100011
6   00000000100000010001001010000011
7   11111110000000101001100011100011
```

Figura 3. Arquivo de Saida

3. Funcionamento do Montador

O montador desenvolvido utiliza duas estruturas principais de dados: os dicionários `REGISTRADORES` e `INSTRUCOES`. O dicionário `REGISTRADORES` associa cada registrador da arquitetura RISC-V à sua respectiva representação binária de 5 bits (por exemplo, `x1 = 00001`, `x5 = 00101`). Já o dicionário `INSTRUCOES` armazena as informações necessárias para a codificação de cada instrução suportada, como o tipo da instrução, os campos *funct3*, *funct7* e o *opcode*. A Figura 4 apresenta essas estruturas.

```python
# REGISTRADORES DICIONARIO
REGISTRADORES = {f'x{i}': format(i, '05b') for i in range(32)} # Cria os registradores (Binarios de 5 bits) de forma automatica


# INSTRUÇÕES DICIONARIO
INSTRUCOES = {
    'add':  {'tipo': 'R',  'funct7': '0000000', 'funct3': '000', 'opcode': '0110011'},
    'or':   {'tipo': 'R',  'funct7': '0000000', 'funct3': '110', 'opcode': '0110011'},
    'sll':  {'tipo': 'R',  'funct7': '0000000', 'funct3': '001', 'opcode': '0110011'},

    'lh':   {'tipo': 'I',  'funct3': '001', 'opcode': '0000011'},
    'andi': {'tipo': 'I',  'funct3': '111', 'opcode': '0010011'},

    'sh':   {'tipo': 'S',  'funct3': '001', 'opcode': '0100011'},

    'bne':  {'tipo': 'SB', 'funct3': '001', 'opcode': '1100011'}
}
```

Figura 4. REGISTRADORES e INSTRUCOES, ambos dicionários

O processo de montagem é dividido em duas etapas principais de leitura do arquivo de entrada. Na primeira etapa, é utilizada a função `calculador_rotulos`, responsável por percorrer o arquivo `.asm` e identificar os rótulos presentes, armazenando-os em um dicionário juntamente com suas respectivas posições de memória. Essa etapa serve para que as instruções de desvio funcionem adequadamente (Figura 5).

```python
# FUNCOES LEITORAS - funcoes que fazem a leitura do arquivo entrada
def calculador_rotulos(arquivo): # Retorna um dicionario de todos os rotulos de um arquivo + as suas posicoes

    rotulos = {} # Cria um dicionario vazio para armazenar todos os rotulos do arquivo
    pos = 0     # O dicionario guarda o rotulo e a posicao da sua instrucao Ex: 'L1' -> 8, 'L2' -> 24, etc

    # pos é contado de 4 em 4 para corresponder ao offset das instrucoes bne

    with open(arquivo) as f:
        for linha in f:
            linha = linha.strip() # Remove espacos e quebra linhas
            if not linha: # Pula caso linha vazia
                continue

            if ':' in linha: # Caso exista um rotulo na linha
                nome, resto = linha.split(':', 1)  # Separa o rotulo do restante da instrucao
                rotulos[nome.strip()] = pos         # Guarda o rotulo e sua posicao no dicionario

                if resto.strip(): # Verifica se tem instrucao pos nomeacao do rotulo. Ex: L1: ; ou L1: or x2, x1, x1
                    pos += 4      # Caso sim, avanca para a proxima instrucao


            else: # Linha normal (sem rotulo), avanca para a proxima instrucao
                pos += 4

    return rotulos # Retorna o dicionario de rotulos resultante
```

Figura 5. calculador_rotulos

Na segunda etapa, o arquivo é novamente percorrido, e cada linha é processada pela função `interpretador_linha`. Essa função tem como objetivo identificar o tipo

de instrução presente na linha e extrair seus operandos, retornando os valores necessários para a etapa de montagem, como registradores e imediatos. Quando determinado campo não é utilizado por uma instrução específica, um valor padrão zero é atribuído (Figura 6).

```python
def interpretador_linha(linha, rotulos, pos): # Define como devesse ler uma linha e quais as suas variaveis
    linha = linha.strip() # Reemove espacos e \n
    if not linha: # Ignora linhas vazias
        return None

    if ':' in linha: # Detecta rotulos
        linha = linha.split(':', 1)[1].strip() # Remove o rotulo e fica apenas com a instrucao
        if not linha: # Caso pos o rotulo nao tenha instrucao. Ex: L1: 'nada'
            return None

    linha = linha.replace(',', ' ') # Prepara a linhas add x1, x2, x3 -> add x1 x2 x3
    partes = linha.split() # Divide a linha em uma lista ['add', 'x1', 'x2', 'x3']

    instr = partes[0]

    rd = rs1 = rs2 = imed = '0' # Padroniza os valores para evitar erros caso algum nao seja usado


    # Os if/elif abaixo servem para ler de forma diferente uma linha a depender de sua instrucao
    # Tipo R: add, or, sll
    if instr in ['add', 'or', 'sll']:
        rd  = partes[1]
        rs1 = partes[2]
        rs2 = partes[3]

    # Tipo I: andi
    elif instr == 'andi':
        rd   = partes[1]
        rs1  = partes[2]
        imed = partes[3]

    # Tipo I (load): lh
    elif instr == 'lh':
        rd = partes[1]
        offset, reg = partes[2].split('(')
        rs1 = reg.replace(')', '')
        imed = offset
```

Figura 6. interpretador_linha

E na função principal, os dados retornados pelo interpretador são utilizados pela função `montador`, responsável por gerar a instrução final em formato binário. Essa função consulta o dicionário `INSTRUCOES` para identificar o tipo da instrução e, com base nisso, organiza os campos de bits de acordo com o formato correspondente (R, I, S ou SB). Para auxiliar nesse processo, são utilizadas funções específicas para a conversão de registradores e valores imediatos, como `conversor_registrador`, `conversor_imediato` e `conversor_imediato_branch`. A Figura 7 e 8 apresenta a implementação da função `montador` e o cabeçalho das funções auxiliares.

Por fim, após o processamento de todas as instruções, o programa verifica se foi especificado um arquivo de saída. Caso positivo, o resultado da montagem é armazenado em arquivo; caso contrário, as instruções binárias são exibidas diretamente no terminal. A Figura 9 apresenta a função principal do programa.

```python
# MONTADOR - Logica principal do programa
def montador(instrucao, rd, rs1, rs2, imediato):
    dados = INSTRUCOES.get(instrucao)
    if not dados:
        raise ValueError(f"Instrução não suportada: {instrucao}")

    tipo = dados['tipo']

    if tipo == 'R':
        return ( # Retorna a Instrucao em seu formato binario final
            dados['funct7'] +
            conversor_registrador(rs2) +
            conversor_registrador(rs1) +
            dados['funct3'] +
            conversor_registrador(rd) +
            dados['opcode']
        )

    elif tipo == 'I':
        n_imediato = conversor_imediato(imediato) # Transforma o imediato em binario

        return ( # Retorna a Instrucao em seu formato binario final
            n_imediato +
            conversor_registrador(rs1) +
            dados['funct3'] +
            conversor_registrador(rd)+
            dados['opcode']
        )

    elif tipo == 'S':
        n_imediato = conversor_imediato(imediato) # Transforma o imediato em binario

        imediato_11_5 = n_imediato[:7] # Bits do imediato da posição 5 até a 11
        imediato_4_0  = n_imediato[7:] # Bits do imediato da posição 0 até a 4

        return ( # Retorna a Instrucao em seu formato binario final
```

Figura 7. montador

```python
# FUNCOES EXTRAS - funcoes auxiliares para a logica do programa
def conversor_registrador(reg): # Retorna o valor binario do registrador procurado···

def conversor_imediato(numero): # Recebe um numero inteiro (entre -2048 e 2047) e o devolve em seu formato binario···

def conversor_imediato_branch(offset): # Prepara o imediato para fazer um branch (bne)···
```

Figura 8. cabeçalhos das funções auxiliares

```python
# MAIN - funcao main do programa
def main():
    if len(sys.argv) < 2:
        print("Uso: py montador.py entrada.asm [-o saida.txt] \n" \
                "Ou" \
                "py montador.py entrada.asm")
        return

    entrada = sys.argv[1]
    saida = None

    if '-o' in sys.argv:
        idx = sys.argv.index('-o')
        saida = sys.argv[idx + 1]

    rotulos = calculador_rotulos(entrada) # Pega-se os rotulos primeiro antes de realmente ler o arquivo

    resultado = [] # Lista das instrucoes finais em binario
    pos = 0 # Posicao atual na leitura do arquivo

    with open(entrada) as f:
        for linha in f: # Interpreta linha a linha do arquivo e monta a sua intrucao no formato binario
            linha_preparada = interpretador_linha(linha, rotulos, pos)
            if linha_preparada:
                instr, rd, rs1, rs2, imed = linha_preparada
                binario = montador(instr, rd, rs1, rs2, imed)
                resultado.append(binario)
                pos += 4

    if saida:
        with open(saida, 'w') as f:
            f.write('\n'.join(resultado)) # Evita linha vazia no final do arquivo
    else:
        print('\n'.join(resultado))

# Comeca o Programa
if __name__ == "__main__":
    main()
```

Figura 9. Função principal, a main

4. Conclusão

O desenvolvimento deste trabalho possibilitou a implementac¸a˜o de um montador simpli ficado para a arquitetura RISC-V. E, atendendo aos requisitos propostos pela disciplina, esse sistema e capaz de traduzir instruc¸o˜es em linguagem assembly para sua representac¸a˜o bina´ria correspondente, incluindo o suporte a diferentes formatos de instruc¸a˜o e ao uso de ro´tulos para instruções de desvios.

Ao longo do projeto, foi possivel colocar em pra´tica importantes conceitos de organizac¸a˜o de computadores, como a estrutura das instruc¸o˜es, conversão do codigo assembly para a linguagem de maquina,o calculo de offset, o papel dos registradores e o funcionamento geral de um montador.

O desenvolvimento deste trabalho contribuiu para uma compreensão mais aprofundada do funcionamento de programas em baixo nível nivel, bem como dos conceitos vistos nos Capitulo 1 e 2 do livro-base utilizado na Disciplina Organização de Computadores.

O código desenvolvido foi disponibilizado em repositório público, podendo ser acessado em: `https://github.com/jotinha156/Trabalho1-ccf-252`.

Referências

[1] Waterman, Andrew; Asanović, Krste. *The RISC-V Instruction Set Manual, Volume I: User-Level ISA*. University of California, Berkeley, 2016. Disponível em: `https://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-118.pdf`. Acesso em: 20 abr. 2026.

[2] RISC-V Foundation. *RISC-V Instruction Set Manual, Version 2.2*. 2017. Disponível em: `https://riscv.org/wp-content/uploads/2017/05/riscv-spec-v2.2.pdf`. Acesso em: 20 abr. 2026.

[3] Rodrigues, João; Santana, Pablo. *Trabalho1-ccf-252*. Repositório no GitHub, 2026. Disponível em: `https://github.com/jotinha156/Trabalho1-ccf-252`. Acesso em: 20 abr. 2026.